

TP4 - ES201

Rian Radeck SANTOS COSTA & Qizheng WANG & Takoua BEN AISSA

Mars 2025

1 Profiling

1.1 Q1

Les deux codes ont été exécutés avec les petites entrées.

Classe d'instructions	Dijkstra		BlowFish	
	nombre	%	nombre	%
Load	14786839	26.94	1924204	14.58
Store	4345536	7.92	575636	4.36
Unconditional branch	437984	0.80	11118	0.08
Conditional branch	9381373	17.09	822891	6.24
Int computation	25930994	47.25	9861380	74.73
Float computation	0	0.00	0	0.00
Trap	241	0.00	26	0.00

Table 1: Comparaison des classes d'instructions entre Dijkstra et BlowFish

1.2 Q2

Nous observons que les instructions de calcul entier constituent une proportion majeure des instructions exécutées, surtout pour BlowFish (74.73%), ce qui suggère que l'optimisation de cette catégorie pourrait améliorer la performance. Cependant, pour Dijkstra, une optimisation de la portion des chargements (*load*), qui représente 26.94% des instructions, serait également bénéfique.

1.3 Q3

Pour comparer les comportements entre Dijkstra, BlowFish, SSCA2-BCS, SHA-1 et le produit de polynômes, nous observons ce qui suit :

- **Similitudes:**

- *Computation Heavy*: BlowFish et SHA-1 se concentrent sur des calculs intenses, utilisant majoritairement l'arithmétique entière, typique des algorithmes cryptographiques.
- *Access Heavy*: Dijkstra et SSCA2-BCS effectuent de nombreuses opérations de chargement et de stockage, caractéristiques des algorithmes traitant des structures de données complexes.

- **Divergences:**

- Dijkstra se distingue par un besoin élevé de branches conditionnelles pour la recherche de chemins optimaux, contrairement à BlowFish et SHA-1, à flux d'exécution plus linéaire.
- Le produit de polynômes juxtapose traitement computationnel et accès fréquents en mémoire, soulignant l'importance de l'optimisation des accès mémoire.

2 Evaluation des performances

2.1 Q4

2.1.1 Explication des codes

Notre makefile:

```
all: dijkstra_large.ss dijkstra_small.ss

dijkstra_large.ss: dijkstra_large.c Makefile
    /usr/ensta/pack/simplescalar-3v0d/bin/sslittle-na-ssstrix-gcc -static
    dijkstra_large.c -O3 -o dijkstra_large.ss

dijkstra_small.ss: dijkstra_small.c Makefile
    /usr/ensta/pack/simplescalar-3v0d/bin/sslittle-na-ssstrix-gcc -static
    dijkstra_small.c -O3 -o dijkstra_small.ss

clean:
    rm -rf dijkstra_large.ss dijkstra_small.ss output*
```

Les paramètres d'exécution de sim-outorder qu'on a utilisés: On a écrit un script dans lequel on parcourt une boucle for pour tester différentes tailles de cache L1. Les paramètres se trouvent dans le code ci-dessous.

Et voici un peu de explication du calcul de *nsets*: Le nombre de sets (*nsets*) dans un cache est donné par la formule :

$$nsets = \frac{\text{Taille du cache}}{\text{Taille du bloc} \times \text{Associativité}} \quad (1)$$

Par exemple, pour la Cache L2:

- Taille du cache : 512 KB = 512 × 1024 B
- Taille du bloc : 64 B

- Associativité : 16

$$nsets = \frac{512 \times 1024}{64 \times 16} = \frac{524288}{1024} = 512 \quad (2)$$

Les paramètres d'exécution de sim-outorder:

```
#!/bin/bash

SIM_OUTORDER=/usr/ensta/pack/simplescalar-3v0d/simplesim-3.0/sim-outorder

PRG_SS="dijkstra_small.ss input.dat"
OPTIONS="-issue:inorder true -max:inst 10000"

BPRED="-bpred bimod"
FETCH="-fetch:mplat 8 -fetch:ifqsize 4"
DECODE="-decode:width 2 -issue:width 4 -commit:width 2"
RUU_LSQ="-ruu:size 2 -lsq:size 8"
RESOURCES="-res:ialu 1 -res:imult 1 -res:falu 1 -res:fpmult 1"
ISSUE="-issue:inorder false"

L2_CACHE="-cache:dl2 dl2:512:32:8:1"

for L1_SIZE_KB in 1 2 4 8 16; do
    L1_SIZE=$((L1_SIZE_KB * 1024 / (64 * 2))) # Calcul du nombre de sets
    IL1_CACHE="-cache:il1 il1:${L1_SIZE}:32:2:1"
    DL1_CACHE="-cache:dl1 dl1:${L1_SIZE}:32:2:1"

    OUTPUT_FILE="sim.out_${L1_SIZE_KB}KB"

    echo "Exécution avec L1I = ${L1_SIZE_KB}KB, L1D = ${L1_SIZE_KB}KB"
    $SIM_OUTORDER $OPTIONS $BPRED $FETCH $DECODE $ISSUE $RUU_LSQ $RESOURCES \
        $IL1_CACHE $DL1_CACHE $L2_CACHE $PRG_SS > "$OUTPUT_FILE" 2>&1

    echo "Résultats enregistrés dans $OUTPUT_FILE"
done
```

2.1.2 Résultats des expériences

Ici, ce sont deux tableaux qui contiennent les statistiques de performance (IPC, CPI, il1 miss rate et dl1 miss rate) de l'algorithme Dijkstra et de Blowfish pour Cortex A7 en fonction de la taille du cache L1.

Taille de cache (KB)	IPC	CPI	il1 miss rate (%)	dl1 miss rate (%)
1	0.3237	3.0891	12.90	12.54
2	0.3256	3.0711	12.67	11.43
4	0.3271	3.0569	12.53	9.82
8	0.3429	2.9165	9.46	9.78
16	0.3520	2.8406	6.58	9.74

Table 2: Performances de Dijkstra de A7 en fonction de la taille du cache L1

Taille de cache (KB)	IPC	CPI	il1 miss rate (%)	dl1 miss rate (%)
1	0.3315	3.0162	4.24	12.84
2	0.3326	3.0068	4.07	12.74
4	0.3332	3.0014	3.97	12.71
8	0.3337	2.9969	3.79	12.69
16	0.3353	2.9828	3.27	12.62

Table 3: Performance de Blowfish de A7 en fonction de la taille du cache L1

Dans la suite, on a analysé plusieurs paramètres influencé par la variation de la taille de cache. Le nombre des instructions par cycle (IPC), le nombre de cycles par instruction (CPI) et le missrate de la mémoire cache il1 et dl1.

A l'aide du matplotlib python, on a tracer les courbes de IPC, CPI et de missrate de il1 et dl1.

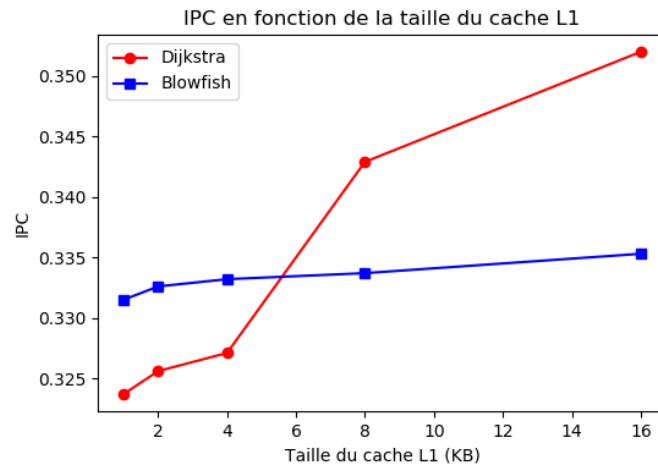


Figure 1: IPC en fonction de la taille du cache L1

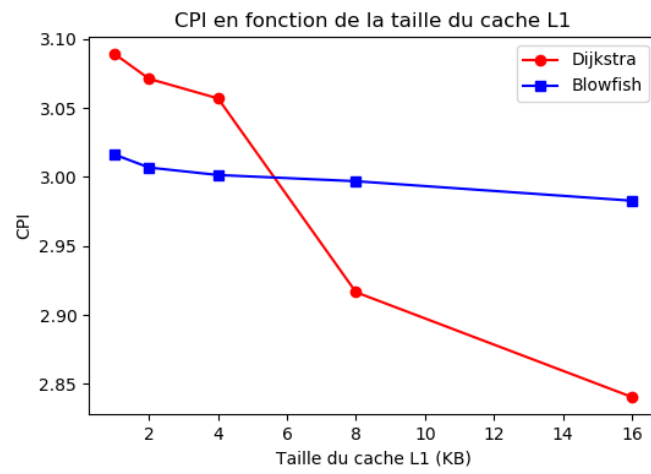


Figure 2: CPI en fonction de la taille du cache L1

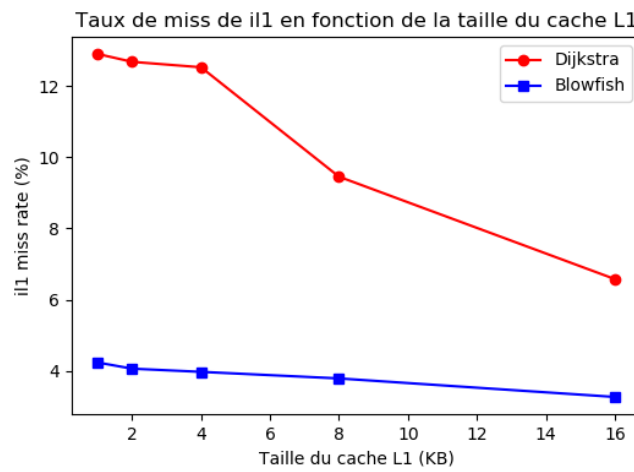


Figure 3: Taux de miss de il1 en fonction de la taille du cache L1

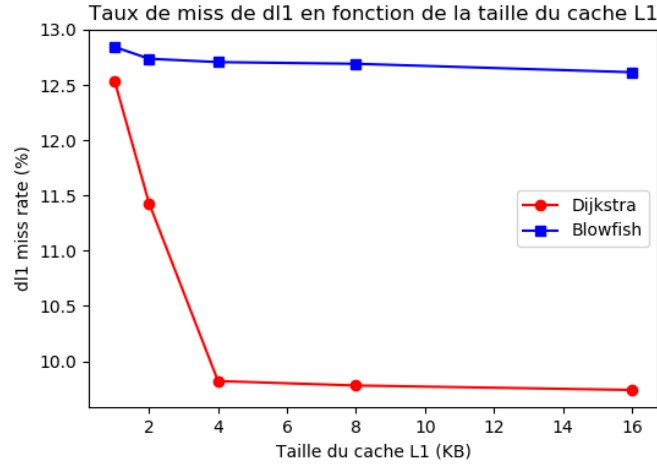


Figure 4: Taux de miss de dl1 en fonction de la taille du cache L1

2.1.3 Analyse des résultats

L'analyse des résultats montre que l'augmentation de la taille du cache L1 améliore les performances de manière significative pour **Dijkstra**, tandis que l'impact est plus faible pour **Blowfish**.

- **Pour Dijkstra** : L'IPC augmente fortement avec la taille du cache, et le taux de miss du i11 diminue drastiquement après 8KB, indiquant que **16KB est la meilleure configuration** pour maximiser l'IPC et réduire les miss.
- **Pour Blowfish** : L'IPC reste relativement stable et le taux de miss de i11 est déjà faible dès 1KB. **Une taille de 4KB à 8KB semble suffisante** pour obtenir de bonnes performances.

Conclusion :

- Meilleure configuration pour Dijkstra : L1 = 16KB
- Meilleure configuration pour Blowfish : L1 = 4KB ou 8KB

2.2 Q5

Dans cette question, on fait la simulation et l'interprétation de la performance du cœur Cortex A15. On va tester encore ces deux algorithmes: DijkstraSmall et Blowfish.

2.2.1 Explication des codes

Notre makefile pour générer le fichier .ss:

```
CC = /usr/ensta/pack/simplescalar-3v0d/bin/sslittle-na-sstrix-gcc
CFLAGS = -O3

SRCS = sha_driver.c sha.c
OBJS = sha_driver.o sha.o
TARGET = sha_driver.ss

all: $(TARGET)

$(TARGET): $(OBJS)
    $(CC) $(CFLAGS) -o $(TARGET) $(OBJS)

# Rule to compile source files into object files
%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    rm -f $(OBJS) $(TARGET)

.PHONY: all clean
```

Les paramètres d'exécution de sim-outorder qu'on a utilisés: On a écrit un script dans lequel on parcourt une boucle for pour tester différentes tailles de cache L1. Les paramètres se trouve dans le code ci-dessous:

```
#!/bin/bash

SIM_OUTORDER=/usr/ensta/pack/simplescalar-3v0d/simplesim-3.0/sim-outorder

PRG_SS="sha_driver.ss input_small.asc"
OPTIONS="-issue:inorder true -max:inst 10000"

# Paramtres fixes du Cortex A7
BPRED="-bpred bimod"
FETCH="-fetch:mplat 8 -fetch:ifqsize 4"
DECODE="-decode:width 2 -issue:width 4 -commit:width 2"
RUU_LSQ="-ruu:size 2 -lsq:size 8"
RESOURCES="-res:ialu 1 -res:imult 1 -res:fpalu 1 -res:fpmult 1"
ISSUE="-issue:inorder false"

# Taille du cache L2 fixe
L2_CACHE="-cache:dl2 dl2:512:32:8:1"

# Boucle pour tester diffrentes tailles de caches L1
for L1_SIZE_KB in 1 2 4 8 16; do
    L1_SIZE=$((L1_SIZE_KB * 1024 / (64 * 2))) # Calcul du nombre de sets
    IL1_CACHE="-cache:il1 il1:${L1_SIZE}:32:2:1"
    DL1_CACHE="-cache:dl1 dl1:${L1_SIZE}:32:2:1"

    OUTPUT_FILE="sim.out_${L1_SIZE_KB}KB"

    echo "Excution avec L1I = ${L1_SIZE_KB}KB, L1D = ${L1_SIZE_KB}KB"
```

```

$SIM_OUTORDER $OPTIONS $BPRED $FETCH $DECODE $ISSUE $RUU_LSQ $RESOURCES
$IL1_CACHE $DL1_CACHE $L2_CACHE $PRG_SS > "$OUTPUT_FILE" 2>&1

echo "Rsultats enregistrs dans $OUTPUT_FILE"
done

```

2.2.2 Résultats des Expériences

Ici, c'est un tableau qui contient les statistiques de performance (IPC, CPI, il1 miss rate et dli miss rate) de l'algorithme Dijkstra et de Blowfish pour Cortex A15 en fonction de la taille du cache L1.

Taille de cache (KB)	IPC	CPI	il1 miss rate (%)	dl1 miss rate (%)
1	0.4684	2.1348	7.79	7.87
2	0.4727	2.1156	7.60	5.69
4	0.4864	2.0561	6.60	5.56
8	0.5038	1.9851	4.38	5.50
16	0.5153	1.9405	3.12	5.35

Table 4: Performance de Dijkstra sur Cortex A15 en fonction de la taille du cache L1

Taille de cache (KB)	IPC	CPI	il1 miss rate (%)	dl1 miss rate (%)
1	0.6288	1.5903	2.42	6.86
2	0.6328	1.5803	2.30	6.83
4	0.6370	1.5697	2.16	6.81
8	0.6454	1.5495	1.84	6.73
16	0.6474	1.5447	1.77	6.69

Table 5: Performance de Blowfish sur Cortex A15 en fonction de la taille du cache L1

A l'aide du matplotlib python, on a tracer les courbes de IPC, CPI et de missrate de il1 et dl1.

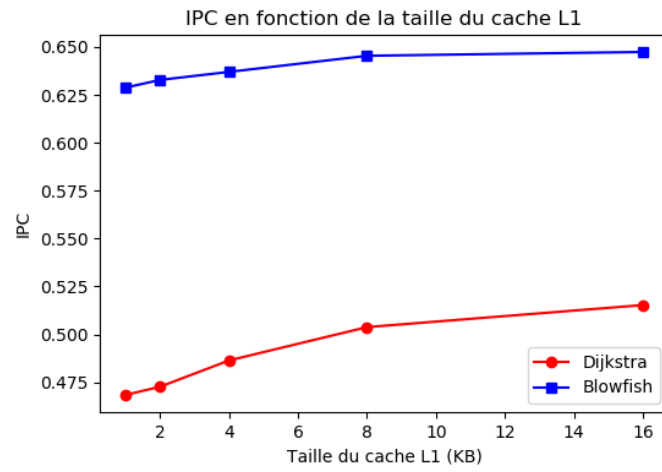


Figure 5: IPC en fonction de la taille du cache L1

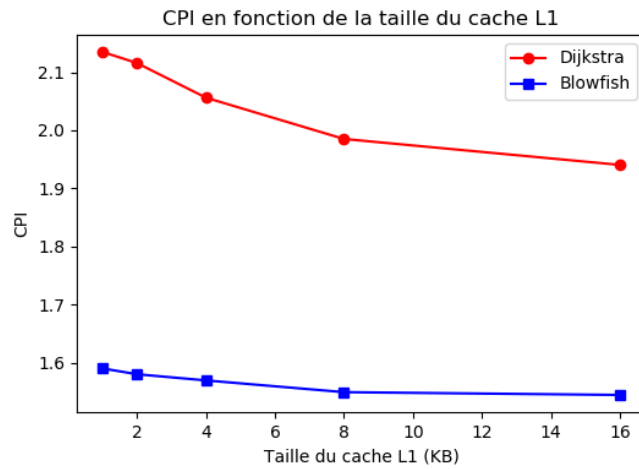


Figure 6: CPI en fonction de la taille du cache L1

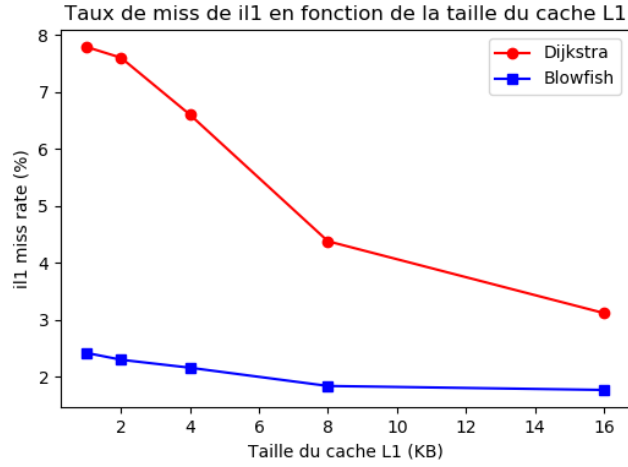


Figure 7: Taux de miss de il1 en fonction de la taille du cache L1

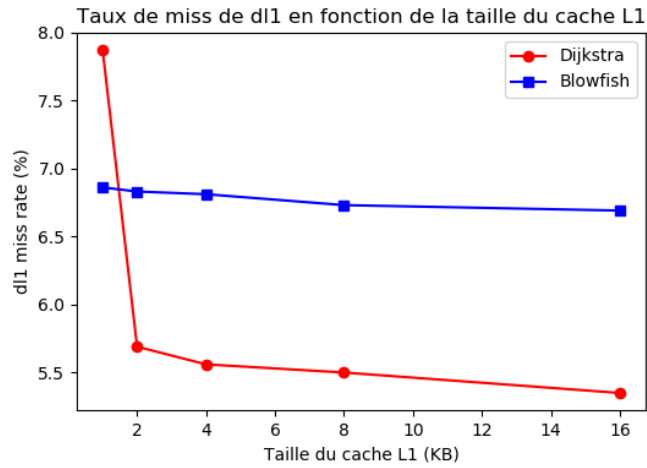


Figure 8: Taux de miss de dl1 en fonction de la taille du cache L1

2.2.3 Analyse des Performances

Les résultats des figures montrent que l'augmentation de la taille du cache L1 améliore les performances générales des applications testées.

- **Pour Dijkstra** : L'IPC augmente avec la taille du cache, tandis que le CPI et les taux de miss diminuent significativement. La meilleure configuration observée est **L1 = 16KB**, où l'IPC est maximal et le taux de miss de il1 est réduit à son minimum.

- **Pour Blowfish** : L'impact de la taille du cache L1 est plus faible, car l'IPC reste relativement stable. Cependant, une taille de cache de **8KB** ou **16KB** permet de minimiser le taux de miss de il1 tout en maintenant des performances optimales.

Conclusion :

- Meilleure configuration pour Dijkstra : L1 = 16KB.
- Meilleure configuration pour Blowfish : L1 = 8KB ou 16KB.

3 Efficacité surfacique

3.1 Q6

En regardant le repo github de Cacti on voit que les paramètres de cache (taille de cache, de bloc, associativité) et la technologie en nm utilisés par défaut sont:

- Cache: -size (bytes) 131072
- Bloc: -block size (bytes) 64
- Associativité: -associativity 2
- Technologie: -technology (u) 0.090

3.2 Q7

En utilisant cette configuration pour le Cortex A15

- -size (bytes) 32768
- -block size (bytes) 64
- -technology (u) 0.028
- -associativity 2

et celle-ci pour le Cortex A7

- -size (bytes) 32768
- -block size (bytes) 32
- -technology (u) 0.028
- -associativity 2

on a obtenu :

- Surface Cache L1 A15 : $0.07152484248 \text{ mm}^2$
- Surface Cache L1 A7 : $0.07151241622 \text{ mm}^2$
- Pourcentage de surface occupé par L1 sur A15 : $\frac{0.07152484248}{2} \times 100 \approx 3.576\%$
- Pourcentage de surface occupé par L1 sur A7 : $\frac{0.07151241622}{0.45} \times 100 \approx 15.891\%$
- Taille du cœur A15 (hors caches L1) : $2 - 0.07152484248 = 1.9284751575 \text{ mm}^2$
- Taille du cœur A7 (hors caches L1) : $0.45 - 0.07151241622 = 0.3784875838 \text{ mm}^2$

Ainsi, les valeurs reflètent une distribution équilibrée des caches L1 au sein de chaque cœur. Pour le Cortex A15, les caches L1 occupent environ 3.576% de la surface totale, ce qui est proportionnellement plus faible comparé au Cortex A7, où les caches L1 occupent 15.891% de la surface. Cela illustre l'importance stratégique des caches L1 dans l'équilibrage entre la surface totale disponible et les performances requises du processeur.

3.3 Q8

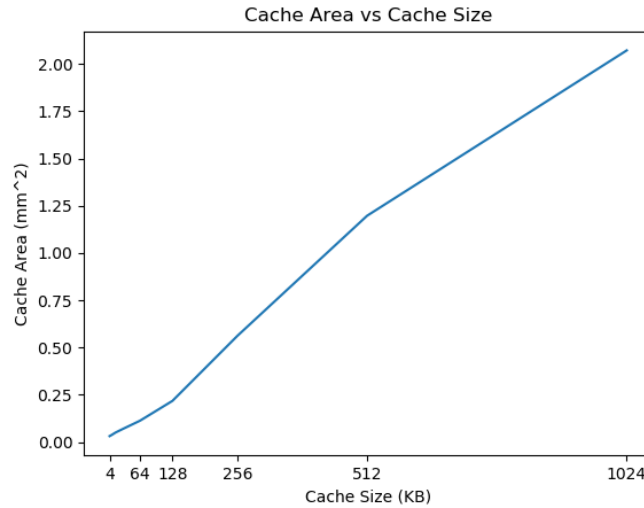


Figure 9: Cortex A15 - Cache Area by L1 Cache Size

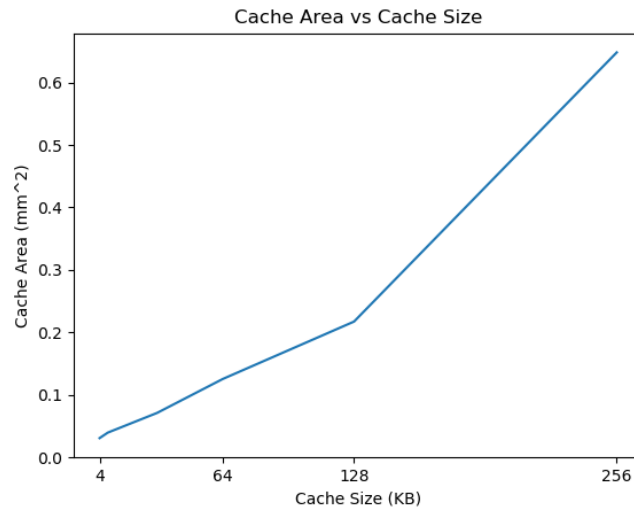


Figure 10: Cortex A7 - Cache Area by L1 Cache Size

Comme on peut le voir, plus la taille du cache est grande, plus d'espace est nécessaire.

3.4 Q9

Il faut juste sommer les tailles de chaque cœur (hors caches L1) aux tailles de cache L1 qu'on a trouvé.

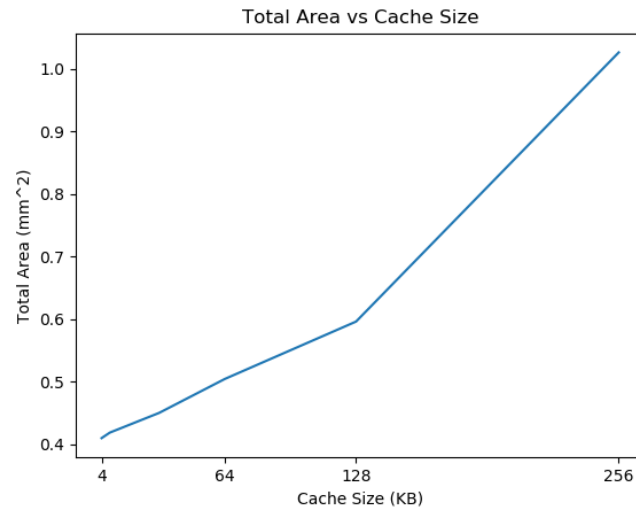


Figure 11: Cortex A7 - Total Area by L1 Cache Size

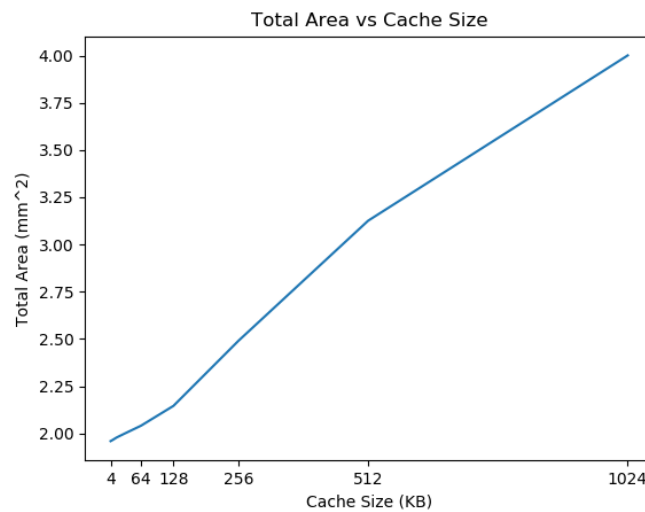


Figure 12: Cortex A15 - Total Area by L1 Cache Size

Ici c'est le code python utilisé pour faire les graphiques.

```
import matplotlib
matplotlib.use('Agg')
```

```

import matplotlib.pyplot as plt
import numpy as np
import subprocess
import re

cache_size = np.array([4, 8, 16, 32, 64, 128, 256, 512, 1024]) * 1024
cache_areas = []

for size in cache_size:
    with open("cache.cfg", "a") as f:
        f.write(f"-size (bytes) {size}\n")

    subprocess.run("./cacti -infile cache.cfg > out.out", shell=True)

    with open("out.out", "r") as f:
        lines = f.readlines()
        print(f"===== {size} =====")
        for line in lines[47:65]:
            print(line.strip())
            match = re.search(r"Cache height x width \(\text{mm}\): (\d+\.\d+) x (\d+\.\d+)", line)
            if match:
                height = float(match.group(1))
                width = float(match.group(2))
                area = height * width
                cache_areas.append(area)
        print(f"=====")
        # input()

    with open("cache.cfg", "r") as f:
        lines = f.readlines()
    with open("cache.cfg", "w") as f:
        f.writelines(lines[:-1])

plt.plot(cache_size / 1024, np.array(cache_areas) + 1.9284751575)
plt.xticks([4, 64, 128, 256, 512, 1024])
plt.xlabel('Cache Size (KB)')
plt.ylabel('Total Area (mm2)')
plt.title('Total Area vs Cache Size')
plt.savefig("out.png")

```

Pour simplicité, on va utiliser les IPC pour l'algorithme Blowfish. pour les prochaines questions.

Table 6: A7 - IPC, Surface et Efficacité Surfactive des Caches

Taille Cache (bytes)	IPC	Surface (mm ²)	Efficacité Surfactive (IPC/mm ²)
1024	0.3315	-	-
2048	0.3326	-	-
4096	0.3332	0.0313	10.64
8192	0.3337	0.0400	8.34
16384	0.3353	0.0505	6.64

Table 7: A15 - IPC, Surface et Efficacité Surfaccique des Caches

Taille Cache (bytes)	IPC	Surface (mm ²)	Efficacité Surfaccique (IPC/mm ²)
1024	0.6288	-	-
2048	0.6328	-	-
4096	0.6370	0.0312	20.41
8192	0.6454	0.0372	17.36
16384	0.6474	0.0505	12.81

4 Efficacité énergétique

4.1 Q10

D'après les données fournies, nous avons calculé la consommation de puissance pour les processeurs Cortex A7 et Cortex A15 à leurs fréquences maximales.

- **Cortex A7 :** À une fréquence maximale de 1.0 GHz, la puissance consommée est de $0.10 \text{ mW/MHz} \times 1000 \text{ MHz} = 100 \text{ mW}$.
- **Cortex A15 :** À une fréquence maximale de 2.5 GHz, la puissance consommée est de $0.20 \text{ mW/MHz} \times 2500 \text{ MHz} = 500 \text{ mW}$.

4.2 Q11

Table 8: A7 - IPC et Efficacité Énergétique des Caches

Taille Cache (bytes)	IPC	Efficacité Énergétique (IPC/W)
1024	0.3315	3315
2048	0.3326	3326
4096	0.3332	3332
8192	0.3337	3337
16384	0.3353	3353

Table 9: A15 - IPC et Efficacité Énergétique des Caches

Taille Cache (bytes)	IPC	Efficacité Énergétique (IPC/W)
1024	0.6288	1257.6
2048	0.6328	1265.6
4096	0.6370	1274
8192	0.6454	1290.8
16384	0.6474	1294.8

5 Architecture système big.LITTLE

5.1 Q12

Afin de choisir la meilleure configuration du cache L1, nous avons priorisé la maximisation de l'efficacité énergétique et surfacique tout en minimisant les ressources. Les simulations des algorithmes Blowfish et Dijkstra montrent des comportements similaires :

Blowfish : L'efficacité surfacique reste relativement constante, mais le Cortex A15 montre une amélioration significative au-delà de 8KB. En revanche, pour le Cortex A7, l'amélioration est plus progressive.

Dijkstra : Même raisonnement que pour Blowfish.

Ainsi, la configuration optimale serait : Cortex A7 L1 8KB / Cortex A15 L1 32KB afin de trouver un équilibre entre performance et consommation.

5.2 Q13

Les configurations proposées pour les Cortex A7 et A15 ne sont pas équivalentes.

Cortex A7 : Très faible consommation énergétique, mais moins performant. Pour Blowfish, les IPC restent sous 0.6 pour une taille de cache entre 1KB et 16KB.

Cortex A15 : Très performant, mais avec une consommation d'énergie élevée. Pour la même plage de cache, le nombre d'IPC peut atteindre 1.8 pour Blowfish et jusqu'à 2.2 pour Dijkstra-small.

Un compromis pourrait consister à utiliser le Cortex A7 pour les applications à faible intensité de calcul et le Cortex A15 pour les charges plus lourdes, avec un cache de taille ajustée selon l'application.

5.3 Q14

Pour spécifier une architecture adaptée à plusieurs applications dans un domaine spécifique, on peut suivre l'approche suivante :

Partir d'une architecture existante pour exploiter une base éprouvée.

Profiling des applications pour identifier les catégories d'instructions les plus critiques.

Ajustement de l'architecture pour trouver un compromis d'amélioration des catégories identifiées.

Simulation de l'impact de la taille du cache sur les performances afin de déterminer la taille optimale.

Simulation de l'impact de la surface du cache sur la consommation d'énergie pour ajuster les ressources.

Choisir la configuration finale qui maximise l'efficacité énergétique et surfacique tout en équilibrant performance et consommation.

Cette approche permettrait de concevoir une architecture hybride qui s'adapte dynamiquement aux besoins des différentes applications, optimisant ainsi les ressources disponibles.